Update Days
Power Platform

#UDPP26

Finding security threats and limiting their impact

Raphael Pothin

Lead Power Platform Reliability Engineer @Manulife

About Me

Raphael Pothin

Microsoft BizApps MVP

Lead Power Platform Reliability Engineer @ Manulife

Power Platform | IaC & Terraform | Security | Developer Experience | Platform Engineering

rpothin on GitHub

Raphael Pothin on LinkedIn

We Covered Attack Paths Here Yesterday

- **GOV-07** showed some approaches attackers can use against Copilot Studio agents
- Prompt injection, data exfiltration, over-privileged actions represent real risks — especially at scale
- **Without strong security posture, most organizations would miss most of these attacks**

> *Power Platform security incidents are often found out about through user reports — or data exposure notices.*

The reality gap

No detection. No telemetry. No response playbook. The attack succeeds and is unnoticed.

Today: the blue-team response

A repeatable, layered workflow for finding threats and limiting their blast radius — starting with observability.

Where Are You Today?

What is your current Power Platform security observability toolbox?

Power Platform Audit Logs
`PowerPlatformAdminActivity`

Power Platform App Insights Logs
`customEvents`

Dataverse Transcripts
`conversationtranscript` , `bots` , `botcomponents`

Defender XDR Advanced Hunting
`AIAgentsInfo` (Preview)

Power Platform Inventory
Environment, agent, and connector inventory visibility

Agents in Microsoft 365 admin center
Centralized agent registry and observability view

Microsoft Purview DSPM for AI
Data security posture insights for agents and AI interactions

Using only 1–2 today? You are in the right place to strengthen your security posture.

The Blue-Team Loop

🔍 Collect → 🎯 Hunt → 🚨 Detect → ⚙️ Automate → 🛡️ Harden

🔍 `Collect`	Establish multi-layer telemetry: Sentinel, App Insights, Power Platform inventory, Dataverse
🎯 `Hunt`	Actively search for risky patterns across all observability layers simultaneously
🚨 `Detect`	Turn validated hunts into scheduled, durable analytics rules
⚙️ `Automate`	Enrich, triage, and contain — at machine speed, via Logic Apps playbooks
🛡️ `Harden`	Feed detection findings back into governance controls and guardrails

Collect

Making agents and their activity visible

🔍 **Collect** → 🎯 Hunt → 🚨 Detect → ⚙️ Automate → 🛡️ Harden

Audit Logs Pipeline to Sentinel

- **Power Platform + Dataverse** emit audit events *(when Dataverse audit is enabled)*
- **Purview Audit** centralizes and normalizes the stream
- **Sentinel Business Apps** ingests and operationalizes for SOC workflows
- This remains the **first collection layer** in our multi-layer workflow

Baseline path: Power Platform → Purview Audit → Sentinel

Research credit: Tomas Prokop (NETWORG)

SIEM ingest essentials

RecordType 261 · `CopilotInteraction` · `CopilotEventData`
Key fields: `UserId` , `CreationTime` , `AppHost`
Use: O365 Management Activity API (`ActivityFeed.Read`)
Scope: `manage.office.com/.default`

Purview schema research highlights

Event shape: `CopilotInteraction` with nested `CopilotEventData`
Resource context: `AccessedResources[]` appears in observed Purview payloads
Model context: `ModelTransparencyDetails[]` captured for interaction analysis

App Insights as Runtime Layer

- App Insights reveals runtime behavior that admin-plane audit logs do not expose
- Track **who/what executed calls** via user-agent and operation patterns
- Baseline **endpoint mix, result codes, and latency buckets**, then trigger Hunt when drift appears

Research credit: Tomas Prokop (NETWORG)

What you can find in App Insights

Operation telemetry: endpoint paths and `operation_Name` trends

Execution context: client type, user-agent patterns, and role/client identity

Health signals: `resultCode`, `duration`, and `performanceBucket`

Strategy impact in the loop

Collect: measurable runtime baselines instead of static visibility

Hunt: prioritize anomalies with endpoint/user-agent novelty and latency drift

Detect: combine runtime anomalies with audit context to reduce false positives

Agents in Microsoft 365 admin center

- **Agent inventory visibility** in Microsoft 365 admin center
- **Security posture checks** per agent (Entra and Purview compliance signals)
- **Operational ownership context** for triage and escalation
- **Fast validation surface** before deep hunting in notebooks

Experience from the admin panel

Single object view: one side panel across `Overview`, `Permissions`, `Data & tools`, `Security`

Lifecycle context: published status, availability, publisher, supported surfaces, and agent type

Operational controls seen in practice

Permissions: OAuth permissions list with admin-consent action, plus Entra ID Protection integration for identity threat signals

Security: direct hooks to Purview for agent activity, sensitive interactions, and AI compliance assessment

Takeaway: triage starts here before deep correlation in Sentinel and notebooks

🎯 Demo Time

Purview DSPM for AI

- **AI observability** centralizes agent activity: total agents, risk level, and sensitive interactions
- **Risk classification** per agent: High / Medium / Low, with interaction types — oversharing, exfiltration, unethical
- **Cross-platform signals**: risk assessed across Purview, Microsoft Defender, and Microsoft Entra
- **Posture objectives** surface highest-priority risks with built-in remediation plans

AI observability key metrics

Total AI apps and agents: active / inactive count across the last 30 days

High-risk agents: broken down by High / Medium / Low risk level

Sensitive interactions: oversharing, exfiltration, and unethical signals per agent

Apps and agents view (Discover)

Per-agent: protection status (*Monitored*), risk level, user / prompts / response trends

Policy coverage: data protection and data compliance policy count per agent

Posture objectives: prevent Copilot data exposure · prevent oversharing

🎯 **Demo Time**

Power Platform Inventory + Dataverse

- **Power Platform inventory from Azure Resource Graph** finds Copilot Studio agents in `PowerPlatformResources`: auth mode, GenAI, publish state, environment
- `bots` + `botcomponents` show live config and risky capabilities: auth drift, email, MCP, HTTP
- `conversationtranscript` adds behavior evidence: silent / spiking volume and suspicious prompt or exfil patterns

Azure Resource Graph inventory signals

Core fields: auth mode, GenAI flag, publish state

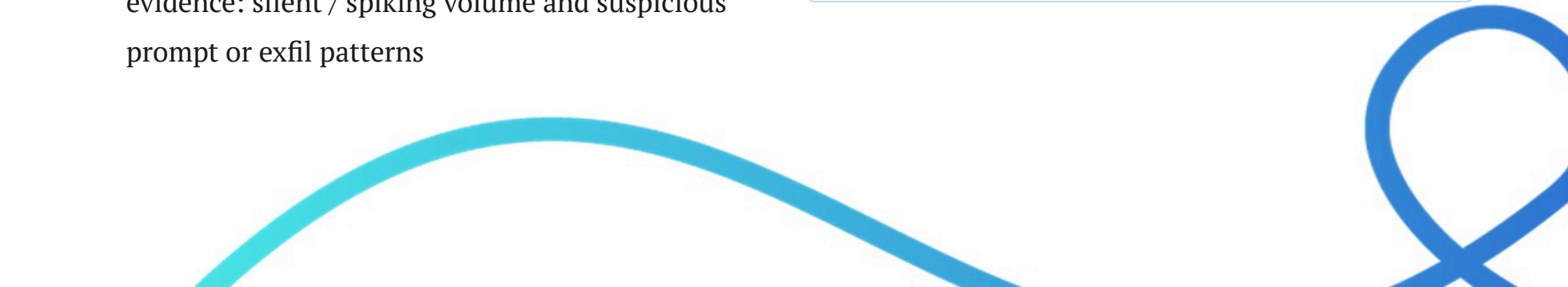
Fast flags: unauthenticated agent, and ARG ↔ `bots` mismatch

Dataverse evidence layer

`bots`: live auth mode and state

`botcomponents`: email, MCP, HTTP capabilities

`conversationtranscript`: volume drift and content scan



Hunt

From assumption to evidence-driven investigation

Collect → **Hunt** → Detect → Automate → Harden

App Insights Has Gone Silent — What Is It Hiding?

The scenario

- 300 req/day agent drops to zero — no alert fires
- Silence is the absence of an event, not an error
- Audit log:
  `ApplicationInsightsConnectionStringUpdated`
  — 3 days ago
- App Insights connection removed, and the agent kept running unobserved

The hunt hypothesis

Telemetry deliberately disconnected. Find: which agent, when, by whom, and what happened during the silence window.

The evidence trail

Sentinel admin activity → disconnect event, actor, environment
`customEvents` baseline → hard drop or full silence (last 3 days)
ARG + Dataverse → auth drift, risky components, transcripts during the gap

The Multi-Layer Hunting Approach

1 Notebook 01 — App Insights Telemetry Drop

Queries `PowerPlatformAdminActivity` + `customEvents` in Sentinel Data Lake

Outputs: telemetry baselines, hard-drop flags, disconnect correlation, suspect environment IDs

2 Notebook 02 — Dataverse Agent Deep-Dive

Queries ARG inventory + Dataverse (`bots` , `botcomponents` , `conversationtranscripts`)

Outputs: auth drift, risky components, transcript content analysis, cross-source risk scoring

3 Hunt Reporter Agent

AI-powered analysis of Notebook 01 + 02 outputs

Outputs: layered Markdown report — Executive Summary, Analytical Narrative, Technical Deep Dive, Remediation Table

🎯 Demo Time

Detect

Automate detection of confirmed potential threats

From Hunt to Detection

A hunt is a question you ask once. A detection is the same question asked automatically, forever.

Validate Confirm the finding is a real pattern. Check historical data for prior occurrences.

Encode Translate the KQL query into a Sentinel Analytics Rule YAML with severity and MITRE tactics.

Schedule Set `queryFrequency` and `queryPeriod` to match the attack's time-to-detect.

Test Replay known-bad data. Confirm true positives fire, and false positives don't flood the queue.

Deploy Commit rule YAML to source control. Deploy via Content Hub or Bicep/Terraform pipeline.

Iterate Alert fatigue is feedback. Tune thresholds until signal-to-noise is acceptable.

Built-in Sentinel Content for Power Platform

Analytics Rules

~40 rules: Dataverse exfiltration and mass operations, Dynamics 365 F&O fraud, Power Apps/Automate lifecycle, Power Platform admin (DLP, connectors, Entra roles)

Hunting Queries

Dataverse access anomalies (post-Entra-alert, cross-environment exports, USB exfil, failed logon follow-up) and Power Apps sharing anomalies

Workbooks

One workbook ships: **Dynamics 365 Activity**. No Power Platform-specific dashboard — you build that.

Playbooks

Dataverse user blocklisting (alert-triggered + Teams/Outlook), SharePoint site management, manager notifications, Teams incident alert

> *Admin-plane coverage is solid.* ***No Copilot Studio rules, no App Insights rules, no workbook for Power Platform*** *— that gap is yours to close.*

Detections as Code: Rules + Notebook Jobs

Not equivalent, but complementary: rules are your always-on alerting engine, while notebook jobs are your deep-analysis engine.

Scheduled Analytics Rules (KQL)

Best for frequent, deterministic detection that creates Alerts/Incidents in Sentinel.

```
kind: Scheduled
queryFrequency: 1h
queryPeriod: 14d
query: |
  customEvents
  | where TimeGenerated > ago(14d)
  | where cloud_RoleName has "CopilotStudio"
  | summarize DailyCount=count()
      by cloud_RoleName,
         bin(TimeGenerated, 1d)
  | where DailyCount == 0
triggerThreshold: 0
```

Notebook Jobs (Jupyter in Sentinel Data Lake)

Best for heavy lifting: long-term analysis, Python enrichment/ML, and writing to custom tables.

Execution	On-demand or scheduled runs from VS Code, with Spark sized per workload.
Data Scope	Correlate analytics + lake tiers for long-retention investigations.
Output	Write enriched outputs to custom tables, while scheduled rules create incidents.

Automate

From passive alerts to active response

🔍 Collect → 🎯 Hunt → 🚨 Detect → ⚙️ **Automate** → 🛡️ Harden

More Rules ≠ Better Security

⚠️ The alert fatigue problem

- Every detection rule adds another incident to triage.
- Without automation, SOC throughput becomes the bottleneck.
- High-severity alerts are drowned in medium-severity noise.
- Analysts start skipping Power Platform incidents as false positives.

The Hunt Report handoff

Structured JSON output (risk score, evidence chain, containment recommendation, severity) is the automation trigger input — not a raw alert.

The goal

Automation handles enrich, score, notify, contain. Analysts focus on context, judgment, and escalation.

> Detection increases volume. Automation restores analyst focus.

Playbook Architecture: Enrich → Triage → Contain

🔬 Enrich

Pull agent metadata, owner identity, environment tier, sensitivity exposure, UEBA risk score

🎯 Triage

Score severity, notify owner via Teams Adaptive Card, and await acknowledgement (15-min SLA)

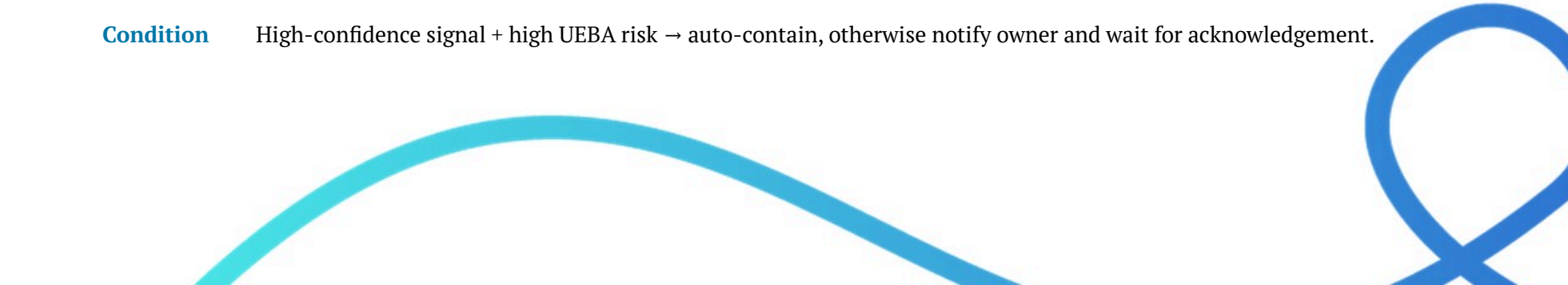
🛑 Contain

Disable agent, isolate environment, open policy review item, update Sentinel incident

Trigger Automation rule on **incident created/updated** (recommended model) for Medium+ Power Platform incidents.

Logic Apps Azure Logic Apps playbook with **Microsoft Sentinel incident trigger** + Power Platform + Teams connectors.

Condition High-confidence signal + high UEBA risk → auto-contain, otherwise notify owner and wait for acknowledgement.



Power Platform Quarantine Strategy Example

Suspicious activity detected on a Copilot Studio agent: deep inspection followed by automated containment if confidence score is high

```
# Quarantine agent (primary contain action)
POST /copilotstudio/environments/[ENVIRONMENT-ID]/bots/[BOT-ID]
  /api/botQuarantine/SetAsQuarantined?api-version=2022-03-01-preview
```

When	Contain branch fires when confidence score is high (for example, score ≥ 8).
Why	`SetAsQuarantined` isolates the agent rapidly, while it remains visible to makers but can't be used.
Confidence Inputs	Dataverse signals can enrich the score: `bots` auth mismatch, risky `botcomponents`, transcript risk hits.

Harden

Pivoting from reactive detection to proactive prevention

🔍 Collect → 🎯 Hunt → 🚨 Detect → ⚙️ Automate → 🛡️ **Harden**

Close the Path, Not Just the Alert

The hunt confirmed it — the evasion path ran through a permission that should never have existed.

- **Root cause** → A maker held the Environment Maker role in production and removed the App Insights connection string directly from Dataverse — no pipeline, no approval, and no alert until you knew where to look.
- **Microsoft ALM guidance is unambiguous** → "App makers and developers shouldn't have access, or should only have user-level privileges" in production environments.
- **The three-part structural fix** → Remove Environment Maker role from makers in production, bind the environment to a restricted Entra security group, and route all deployments through a pipeline service principal.
- **A service principal properly configured is harder to manipulate** → It has a scoped role, no inbox, and no ability to bypass the deployment pipeline. The maker is removed from the blast radius entirely.

The flywheel pays off here — Harden is where evidence becomes structure. The loop gets stronger only if each cycle ends with a permission removed, a gate added, or a policy enforced in code.

Key Takeaways

Collect across layers
Correlate Sentinel audit, App Insights, inventory, and transcripts for full attack context.

Hunt silence as a signal
Treat telemetry drops as suspicious and verify disconnects, runtime gaps, and config drift.

Promote hunts into detections
Convert proven investigations into scheduled rules and notebook jobs that run continuously.

Automate with clear criteria
Enrich, triage, and quarantine fast on high confidence, and keep analysts focused on decisions.

Harden root causes in code
Tighten risky permissions and codify controls so each detection cycle reduces blast radius.

Update Days
Power Platform

#UDPP26

Thank You!

RaphaelPothin

Raphael POTHIN

rpothin

Key Resources

🔬 Demo Repository

github.com/rpothin/udpp26-finding-security-threats

Notebooks, KQL templates, Logic App YAML, Terraform DLP modules.

📖 Sentinel Business Apps Solution

learn.microsoft.com/en-us/azure/sentinel/business-applications/solution-overview

🔗 Connect Power Platform to Sentinel

learn.microsoft.com/en-us/azure/sentinel/business-applications/deploy-power-platform-solution

🤖 Copilot Studio Security

learn.microsoft.com/en-us/microsoft-copilot-studio/security-and-governance

🏛️ PPCC25 Companion (Origin Story)

github.com/rpothin/ppcc25-finding-security-threats

Predecessor session material from Power Platform Community Conference 2025.

Update Days
Power Platform

#UDPP26

Session: #18

GIVE US FEEDBACK FOR THIS SESSION

Finding security threats and limiting their impact